

Document 03 — LLM Basics

This is for self-study and live mentoring. Full plan: [CURRICULUM.md](#)

Prerequisites

[02_apis_http_json](#)

How to Read & Practice This Document

- **What:** how a language model actually works when it's answering you.
- **Why:** without this, every prompt you write is a guess, and every wrong answer feels like a random mystery instead of something you could have expected.
- **When:** before writing any prompt that matters — every decision about cost, length, and randomness comes back to this document.
- **How to practice:** 1. Read the material once, just to get the shape of it. 2. Do the **Basic** guessing exercises closed-book, then check yourself against the real tokenizer tool. 3. Do the **Intermediate/Real-world** exercises, and compare your guesses to what actually happens. 4. There's no coding in this document — the goal is a well-tuned gut feeling. Test it: guess first, then check. 5. Stuck on an idea, not code? Ask for **Hint 1** through **Hint 4** — the hint system works for ideas too, not just code. 6. Before moving on, explain out loud why a model making things up isn't a "bug" the same way a crash is. If you can't, you're not done.

Jump to: [Core Concepts](#) · [Practice Exercises](#)

The Story — what this document is actually building

Doc02 taught you that the network can fail in specific, predictable ways. This document is about a different kind of unpredictability — the model itself, and what actually happens inside a single call to it. There's no code to build here, but everything you learn becomes a gut feeling you'll use in every prompt you write from Doc04 onward.

First, the model never sees your words the way you do — it sees **tokens**, small chunks of text turned into numbers. Every limit and every price is measured in tokens, not words, so "how long is my prompt" is really "how many tokens is my prompt."

Second, everything you send and everything the model sends back shares one **context window** — a shared budget, not a one-way "how much can I paste in." And because the model has **no memory** of its own, every earlier turn of a conversation gets resent, and re-paid for, on every new call — which is also why a long conversation eventually risks the context window itself.

Third, the model doesn't just spit out one fixed answer — at each step it picks from a list of likely next tokens, and **temperature** controls how randomly it picks. Low temperature for tasks that need the same answer every time, higher temperature for tasks where variety is good.

Fourth, since input and output tokens are priced differently, and a long resent history multiplies both, cost is something you can actually estimate in advance, before you spend a cent.

Fifth, a model **making things up** isn't a bug you can patch — it's predicting "what sounds like a natural next word," with no built-in way to check that against the real world. Knowing this changes how you react to it (mitigate, don't expect to eliminate).

That's the story: tokens, context windows, memory, temperature, cost, and hallucination are one connected picture of what actually happens when you call a model — not five separate trivia facts. There's no Build Task here because this document's job is intuition, not code — Doc04 is where you start writing the calls this intuition will guide.

Core Concepts (read this first — everything you need is here)

Tokens: what the model actually reads

A token is not a word — it's a small chunk of text (sometimes part of a word, sometimes a whole word, sometimes just punctuation) that the model turns into a number. "ChatGPT" might be one token; "unbelievably" might split into two or three pieces. As a rough guide, 1 token is about 4 characters of English text, or about $\frac{3}{4}$ of a word — but this is just a rule of thumb, not a fixed rule, and it changes for other languages (non-English text often uses more tokens for the same meaning).

Why this matters: every limit you'll hit — how much text fits, how much something costs — is measured in tokens, not words or characters. So thinking in "word count" will mislead you, especially with code, non-English text, or unusual spacing. **How it works:** the model was trained on a fixed list of token patterns. Text gets split into that list before it ever reaches the actual neural network — the network only ever sees numbers, never letters.

Context window: a shared budget, not just "how much you can paste in"

The context window is the largest number of tokens a model can handle in one call — and importantly, **the text you send in and the reply it gives back share this same budget**, along with the whole conversation history you resend every turn. **Why this is easy to get wrong:** it's tempting to think "how long a document can I paste in," but the real limit is: everything you send + everything you expect back + everything already in the conversation, all added together. **What happens if you go over it:** depending on the tool you're using, you'll either get a clear error, or (in some setups)

some of the text just gets silently cut off — which is worse, because your code seems to work while quietly losing information. **How to think about this in practice:** a long conversation doesn't just risk getting cut off — it also means every earlier message gets resent (and you pay for it again) on every new call, since the model doesn't remember anything on its own, which connects to the next idea.

No memory: there's nothing "remembered" inside the model

Every API call to a chat model stands completely alone — the model has no memory of any earlier call, unless you send that earlier conversation back to it yourself, as part of the new call. **Why:** this is just how the API works — it takes in a set of messages and returns the next message, with no ongoing session kept on the server by default. **What this really means:** any "memory" in a chatbot you build isn't something the model is doing — it's *your code* keeping a growing list of messages and sending the right ones back each time. This is also why cost grows as a conversation gets longer, even if the user keeps typing short messages — you're paying again for the whole history, every single turn.

Randomness: `temperature` and `top_p`

At each step, the model doesn't just pick "the next word" directly — it works out a list of possible next tokens, each with a probability, then **picks one** based on those odds. `temperature` controls how sharp or spread out those odds are: near 0, the model almost always picks the single most likely token (closer to always giving the same answer). Higher values spread the odds out more, making less-likely tokens more likely to get picked, which feels more "creative" or random. `top_p` works a bit differently — it only considers the smallest group of tokens whose combined odds add up to `p`, cutting off the very unlikely options entirely. **Why this matters in practice:** `temperature=0` doesn't guarantee the exact same answer every single time (there's some randomness in the systems behind the API too), but it gets close — use it when you need consistency, like pulling structured data out of text or sorting things into categories. Higher temperature suits open-ended writing, where variety is a good thing, not a problem.

Cost: input and output tokens are priced differently

API pricing is usually a rate for each input token, and a separate, usually higher, rate for each output token — generating new text costs more than reading existing text. **Why this matters for how you design things:** a system prompt you send on every call (input) is cheaper per token than the model's replies (output), but if it's long and you send it thousands of times, it adds up — this is exactly the problem that prompt caching (covered again in Doc12) solves. **How to roughly estimate cost:** $(\text{input tokens} \times \text{input rate}) + (\text{output tokens} \times \text{output rate})$, added up across every call in a session — and remember that in a back-and-forth conversation, "input tokens" includes the *entire* resent history, not just the newest message.

Making things up: not a bug, just how the model works

A language model writes text by predicting the most likely next word given everything before it — it has no built-in way to check "is this actually true" against the real world. It only checks "does this sound like a natural continuation of this text." **Why this means you can't just patch it away:** a confident, fluent, wrong answer is produced by the exact same process as a confident, fluent, correct one — the model has no internal flag telling them apart. **When it gets worse:** when there wasn't much training data on a topic, when you ask for very precise facts (exact dates, numbers, sources), or when you push the model to reason beyond what naturally fits in front of it. **How real systems deal with this** (previewed here, built properly in Doc08 and Doc13): grounding answers in real, retrieved documents (RAG), so the model has actual source text to work from, and testing setups that catch answers not backed by any source — but neither one fully *removes* the underlying cause, they just make it show up less often.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- [OpenAI Tokenizer \(interactive tool\)](#) — paste in text, see it split into tokens.
- [tiktoken \(OpenAI's tokenizer library\)](#) — count tokens in code, not just in the browser tool.
- [OpenAI — Prompt engineering guide](#) — practical guidance straight from OpenAI.
- [Anthropic — Building Effective Agents](#) — not tied to one company, and very good on what LLMs are good and bad at. Worth reading again after Doc11.

Practice Exercises (no coding this document — guessing and estimating)

Why no code here: this document is about building correct intuition (tokens, cost, context limits), not a coding skill on its own — the code that uses this understanding starts in Doc04. If you want to run the token-counting exercise below in real code instead of the browser tool, `pip install tiktoken` and use it directly — that's the one optional script worth writing here.

Jump to an exercise: [Basic](#) · [Intermediate](#) · [Real-world](#) · [Edge cases](#) · [Failure](#)

Basic — guess token counts, then check

- **What:** guess the token count of 5 sentences of different lengths, then check with the tokenizer tool.
- **Why:** you need your own gut feeling for tokens before Doc04, because every cost and context-limit decision from here on depends on it.
- **When you'll hit this for real:** every time you write a system prompt and wonder "is this going to be expensive," before you've written a single line of code to check.

- **How to practice it:** open platform.openai.com/tokenizer, write down your guess for each sentence *before* pasting it in, then compare.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Intermediate — temperature and cost, side by side

- **What:** run one prompt at `temperature=0` and `temperature=1`, three times each, and separately work out the cost of a 2,000-input/500-output-token call.
- **Why:** seeing the actual variation (or lack of it) at each temperature setting is more convincing than reading about it — and cost math you've done by hand once, you'll estimate correctly forever after.
- **When you'll hit this for real:** choosing a temperature setting for a real feature (Project 1's Triage agent needs `temperature=0`; a creative-writing feature wouldn't).
- **How to practice it:** once you have Doc04's client, loop 3 calls at each temperature and print all 6 answers side by side. For cost: look up OpenAI's current per-token prices and multiply by hand.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Real-world — how long can this conversation actually go?

- **What:** guess the largest number of conversation turns that fit in a given context window, for an average message length you choose.
- **Why:** this is the exact question behind Project 1's "conversation silently exceeds the context window" failure — you're building the intuition that predicts it before it happens.
- **When you'll hit this for real:** designing any chat feature — you need to know roughly when to start trimming history, before a user's long conversation breaks silently.
- **How to practice it:** pick an average message length (e.g. 50 tokens), pick a context window size (e.g. 128k), and do the division — then sanity-check against a real long conversation if you have one.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Edge cases — where the $\frac{3}{4}$ -word rule breaks

- **What:** guess the token count of a non-English prompt (or one full of code), before checking.
- **Why:** the "1 token $\approx \frac{3}{4}$ of a word" rule of thumb quietly stops working outside plain English — you need to know that *before* it costs you real money on a multilingual or code-heavy feature.
- **When you'll hit this for real:** any project with non-English users, or one where you're sending code (not prose) to the model — both tokenize far less efficiently than English text.
- **How to practice it:** take one prompt in Urdu (or another non-English language) and one snippet of Python code, guess both token counts, then check both against the tokenizer tool. Note how far off you were, for each.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Failure — make it confidently wrong, on purpose

- **What:** write a prompt you're confident will make the model state something false, confidently.
- **Why:** doing this once, deliberately, is what turns "the model hallucinated" from a mysterious, alarming event into an expected, structural failure mode you can explain to someone else.
- **When you'll hit this for real:** the first time a user reports a wrong answer from your app, and you need to explain *why* it happened, not just apologize.
- **How to practice it:** ask about a very specific, obscure fact (an exact statistic, a niche detail) the model likely wasn't trained on well. Write one paragraph explaining, in your own words, why it happened — without using the word "mistake."
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Expected Behavior / What You Should Be Able to Predict

- Before running a prompt, you should be able to roughly guess: how many tokens it uses, roughly what it will cost, and whether it risks going over the context limit.
- You should be able to guess, without running it, whether raising the temperature will actually change a given prompt's answer much (some prompts barely change no matter the temperature; others change a lot).

Break-It / Debug Preview

- A prompt that quietly goes over the context limit (what actually happens — does it get cut off? does it error? which part gets cut?).
- A `temperature=1` call used somewhere that needs the same answer every time.
- Full debugging drill in [14_debugging_lab](#).

Interview Topics Preview

- Why LLMs have no memory of their own · what a context window actually limits · why cost isn't simply "how much I asked for" · temperature vs. top_p · why making things up can't just be "fixed" like a bug.

Move On When

You can guess, before running it, roughly how a prompt will behave and roughly what it will cost. Full details: [CURRICULUM.md §4](#).

Stuck? Ask for **Hint 1** or **Hint 2** (each has Basic/Intermediate/Advanced depth). Ask for the full explanation only if you say "**Show me the solution.**"